UTEC Universidad Tecnológica	**INGENIERÍA MECATRÓNICA**	
	Unidad Curricular: Tecnologías de Microprocesamiento	
	Semestre: IV	
	Docentes: Jesús Martínez / Marcelo Díaz	
	4to Laboratorio	

Fecha:___ /___

Nombre y Apellido:		**C.I.:**	
Nombre y Apellido:		**C.I.:**	
Nombre y Apellido:		**C.I.:**	

Objetivo del Laboratorio:

En esta cuarta instancia de laboratorio se busca profundizar en el uso del microcontrolador ATmega328p y de sus periféricos en el desarrollo de sistemas embebidos. Se trabajará en la implementación de aplicaciones prácticas que involucren comunicación RFID, SPI y Bluetooth, lectura de sensores, control de actuadores, manejo de pantallas e indicadores visuales y sonoros, así como el almacenamiento persistente de datos mediante EEPROM.

Consideraciones y Pautas de trabajo:

1. Entregar evidencias de cada una de las etapas desarrolladas durante la resolución de los problemas planteados.
2. Los programas deberán desarrollarse utilizando lenguaje C para el ATmega328P, respetando la sintaxis y las instrucciones correspondientes a la arquitectura AVR.
3. Consultar la hoja de datos (datasheet) del ATmega328P para comprender la organización de sus registros, puertos, bits de configuración y demás recursos utilizados durante el laboratorio.
4. Simular todos los programas para comprobar su correcto funcionamiento antes de pasar a la implementación física.
5. Presentar a los docentes el funcionamiento de los programas desarrollados y explicar las decisiones tomadas durante su implementación.
6. Grabar y entregar un video demostrando el correcto funcionamiento de las actividades.
7. Antes de realizar cualquier conexión física, verificar la alimentación del microcontrolador, las conexiones de VCC y GND, los niveles de tensión y la configuración de los pines utilizados.

Problema A - Cajero Automático con Microcontrolador ATmega328P, Lector RFID RC522 y Tarjetas MIFARE

El proyecto consiste en la emulación de un cajero automático (ATM) que utiliza un lector RFID RC522 para identificar tarjetas MIFARE a través de su UID, un teclado matricial **4x4** para introducir el PIN y los montos de las operaciones, una pantalla LCD **16x2** para mostrar mensajes de bienvenida e interacción, y dos LEDs (verde y rojo) más un buzzer para indicar el resultado de cada operación.

El sistema administra **3 cuentas distintas**, cada una asociada a una tarjeta MIFARE identificada por su UID. Para cada cuenta se almacena en la memoria EEPROM del ATmega328P:

- El **UID** de la tarjeta (4 bytes).
- El **PIN** de 4 dígitos.
- El **saldo** disponible (en unidades monetarias enteras).

El usuario puede **retirar** o **depositar** dinero, y el saldo se actualiza de forma persistente en la EEPROM, por lo que se conserva aunque el sistema se apague o reinicie.

Funcionalidades principales

1. **Identificación por UID:** el RC522 lee el UID de la tarjeta MIFARE y lo compara con los UIDs almacenados en EEPROM.
2. **Autenticación con PIN:** cada cuenta tiene un PIN de 4 dígitos guardado en EEPROM. El usuario lo ingresa por el teclado.
3. **Consulta de saldo:** el sistema muestra el saldo actual de la cuenta.
4. **Retiro de dinero:** el usuario ingresa un monto; si hay saldo suficiente, se descuenta y se actualiza la EEPROM.
5. **Depósito de dinero:** el usuario ingresa un monto y se suma al saldo, actualizándose la EEPROM.
6. **Persistencia:** los UIDs, PINs y saldos se conservan tras apagar o reiniciar el sistema.
7. **Indicadores:** LED verde en operación exitosa, LED rojo en error, buzzer tras 3 intentos fallidos de PIN.
8. **Pantalla LCD:** guía al usuario paso a paso durante toda la interacción.
9. Implementar una Máquina de estados

Problema B - Simulador de Proceso Industrial con Banda Transportadora usando 2× ATmega328P en Arquitectura Maestro–Esclavo SPI

El proyecto consiste en la emulación de una línea de producción industrial simplificada, compuesta por una banda transportadora, un pistón que actúa

sobre las piezas y un sistema de señalización y paro de emergencia. El sistema se implementa con **dos microcontroladores ATmega328P** que se comunican mediante **SPI** bajo una arquitectura **maestro–esclavo**:

- El **maestro** se encarga de la **toma de decisiones**, la lectura de sensores y la **interfaz con el usuario** (LCD + botones + potenciómetro).
- El **esclavo** se encarga exclusivamente de **ejecutar las órdenes** sobre los actuadores (motor de la banda, servo del pistón, LEDs indicadores y buzzer).

El maestro monitorea en todo momento el estado del proceso (piezas detectadas, temperatura simulada del motor, velocidad seleccionada) y envía comandos por SPI al esclavo, quien los traduce en acciones concretas. El LCD del maestro muestra en tiempo real tanto las lecturas de sensores como los comandos enviados.

Requerimientos

Componente	Descripción	Cantidad
Microcontrolador	ATmega328P (Arduino UNO/Nano)	2
Sensor de pieza	LDR (detecta pieza que pasa por la banda)	1
Sensor de temperatura	DHT11 (simula temp. del motor)	1
Entrada usuario	Botón Start/Stop	1
Entrada usuario	Potenciómetro (velocidad de banda)	1
Pantalla	LCD 16x2 (I2C o paralelo)	1 (maestro)
Actuador	Motor DC con driver L293D/L298N (banda)	1 (esclavo)
Actuador	Servo SG90 (pistón)	1 (esclavo)
Indicador	LED verde (marcha)	1 (esclavo)
Indicador	LED rojo (falla)	1 (esclavo)
Alarma	Buzzer activo	1 (esclavo)
Alimentación	Fuente 5 V / 9 V (motores aparte)	—

Funcionalidades principales

1. **Arranque/paro del proceso:** botón en el maestro envía comandos START / STOP al esclavo.
2. **Detección de pieza:** el LDR del maestro detecta el paso de una pieza y envía el comando PISTON al esclavo, que acciona el servo.
3. **Control de velocidad:** el potenciómetro del maestro ajusta la velocidad (PWM) del motor de la banda en el esclavo.
4. **Monitoreo de temperatura:** el DHT11 del maestro lee la temperatura simulada del motor; si supera un umbral, se envía PARO_EMERGENCIA.

5. **Señalización:** LEDs verde/rojo y buzzer en el esclavo indican marcha normal, falla o emergencia.
6. **Visualización en LCD:** el maestro muestra sensores, contador de piezas y comandos enviados.
7. **Comunicación SPI robusta:** tramas de 2 bytes [COMANDO][VALOR] con confirmación (ACK) opcional del esclavo.

Problema C - Sistema de Alarma Antirrobo con Supervisión Inalámbrica usando 2× ATmega328P y Módulos Bluetooth HC-05

El proyecto consiste en la emulación de un sistema de alarma antirrobo con supervisión y control inalámbrico remoto. Se implementa con **un microcontrolador ATmega328P**, equipado con un **módulo Bluetooth HC-05**, que establecen un enlace serial inalámbrico para intercambiar información de estado y comandos de control con un dispositivo celular.

- El **Nodo Planta (casa)** está acoplado físicamente al sistema vigilado: lee sensores de presencia y apertura, controla la sirena/buzzer local y reporta por Bluetooth su estado.
- El **Nodo Remoto (cuidador)** recibe la información, la muestra en pantalla y permite al operador **armar/desarmar** la alarma, **silenciarla** y **seleccionar el modo AUTO/MANUAL** mediante botones y comandos desde el celular.

El sistema puede operar en dos modos:

- **Modo automático:** el nodo planta gestiona la alarma por sí solo; cualquier evento dispara la sirena y notifica al nodo remoto.
- **Modo manual:** el operador del nodo remoto arma/desarma, silencia o fuerza la alarma desde la distancia.

El celular muestra en tiempo real el **estado del sistema**, la **zona afectada** y los **mensajes enviados/recibidos** por Bluetooth.

Requerimientos

Componente	Descripción	Cantidad
Microcontrolador	ATmega328P (Arduino UNO/Nano)	1
Módulo Bluetooth	HC-05 (maestro-esclavo configurable)	1
Sensor de apertura	LDR (simula ventana/portón abierto)	1
Sensor de presencia	PIR HC-SR501 o botón (simula movimiento)	1
Alarma	Buzzer activo o sirena 5 V	1 (nodo planta)
Indicador local	LED rojo (alarma) + LED verde (armado)	2 (nodo planta)

Componente	Descripción	Cantidad
Alimentación	Fuente 5 V regulada	—

Funcionalidades principales

1. **Enlace Bluetooth bidireccional:** los dos nodos intercambian tramas de texto.
2. **Armado y desarmado remoto:** el operador arma/desarma desde el nodo remoto/celular.
3. **Detección de intrusión:** LDR (apertura) y PIR/botón (movimiento) disparan la alarma local.
4. **Notificación inalámbrica:** el nodo planta informa al remoto del evento (ALARMA:ON;ZONA:VENTANA).
5. **Silenciar alarma:** el operador puede silenciar la sirena local sin desarmar el sistema.
6. **Modo AUTO / MANUAL:** en AUTO la alarma se gestiona sola; en MANUAL el operador decide.
7. **Señalización local y remota:** LEDs y buzzer indican estado y alarma en ambos nodos.
8. **Visualización en LCD:** el nodo remoto muestra estado, zona afectada y últimas tramas.

Problema D - Juego de Memoria de Secuencias con Joystick, Botón y Matriz LED RGB 8x8 sobre ATmega328P

El proyecto consiste en la implementación del clásico juego **"Simón Dice"** adaptado a una matriz LED RGB 8x8. El sistema genera una secuencia creciente de posiciones iluminadas sobre la matriz, y el jugador debe repetirla moviendo un cursor luminoso con un joystick y confirmando cada posición con el botón del joystick (SW).

- El **joystick** permite mover el cursor por la matriz en las cuatro direcciones, respetando los límites físicos del arreglo.
- El **botón del joystick** cumple dos funciones según el estado del juego: confirmar la posición del cursor durante el turno del jugador, y cambiar el color del cursor de forma aleatoria cuando se está en modo libre.
- La **matriz LED RGB 8x8** WS2812B muestra tanto la secuencia generada por el sistema como el cursor del jugador.
- El **buzzer** emite un tono distinto para cada color, reforzando la asociación visual-auditiva típica del juego original.
- El sistema avanza de **nivel** cada vez que el jugador acierta la secuencia completa; si falla, se muestra una animación de error y el juego se reinicia desde el nivel 1.

El estado del juego, el nivel y la puntuación máxima se almacenan en la **EEPROM** del ATmega328P, de modo que el récord se conserva entre sesiones.

Requerimientos

Componente	Descripción	Cantidad
Microcontrolador	ATmega328P (Arduino UNO/Nano)	1
Matriz LED	Matriz RGB 8x8 (WS2812B o módulo con MAX7219 + LEDs RGB)	1
Matriz	controlador integrado	1
Joystick	Módulo joystick analógico con pulsador (KY-023)	1
Buzzer	Buzzer activo o pasivo 5 V	1
Botón extra	Botón de reinicio (opcional)	1
Alimentación	Fuente 5 V regulada (mínimo 1 A si se usan WS2812B)	1
Cables	Jumper y protoboard	varios

Funcionalidades principales

1. **Generación de secuencia aleatoria:** el sistema genera una secuencia de posiciones (fila, columna) y colores que crece en uno por cada nivel superado.
2. **Reproducción de la secuencia:** la matriz ilumina cada posición con su color durante ~500 ms, con pausa de 200 ms entre elementos, acompañada de un tono distinto en el buzzer.
3. **Cursor controlado por joystick:** el jugador mueve un cursor por la matriz con las cuatro direcciones, con **zona muerta** para evitar lecturas erráticas.
4. **Confirmación con el botón:** al presionar el botón del joystick, el sistema verifica si el cursor está en la posición correcta de la secuencia.
5. **Cambio de color aleatorio:** cuando el juego está en modo libre (esperando o entre niveles), el botón cambia el color del cursor de forma aleatoria.
6. **Límites de la matriz:** el cursor no puede salir del rango [0..7] × [0..7].
7. **Detección de acierto/error:**
    - Acierto completo → sube de nivel, animación de éxito y nueva secuencia más larga.
    - Error → animación de fallo (parpadeo rojo), tono grave, y reinicio del juego.
8. **Puntuación y récord en EEPROM:** el nivel alcanzado y el récord histórico se guardan y muestran al reiniciar.

9. **Indicadores visuales y sonoros:** el buzzer emite un tono asociado a cada color, y la matriz muestra animaciones de éxito/error.

Problema E - Burbuja de Inclinación con MPU6050, Botón y Matriz LED RGB 8x8 sobre ATmega328P

El proyecto consiste en un **visualizador de nivel (inclinación)** que representa, sobre una **matriz LED RGB 8x8**, una **burbuja luminosa** que se desplaza en tiempo real según la **inclinación física** detectada por un **sensor MPU6050** (acelerómetro + giroscopio) conectado por **I2C** al ATmega328P.

- La posición del LED activo en la matriz se calcula a partir de los ángulos de inclinación en los ejes X e Y, mapeados al rango de la matriz (0–7).
- El color de la burbuja cambia dinámicamente según el eje predominante de inclinación: **rojo** si predomina X, **verde** si predomina Y, **amarillo** si la inclinación es combinada, y **azul** cuando el sistema está nivelado.
- Un **botón físico** permite **congelar** la posición actual (modo *hold*) y otro (o doble pulsación) permite **recalibrar** la posición neutra del sensor, guardando el offset en la **EEPROM**.
- La matriz tiene **límites físicos**: la burbuja nunca sale del área visible, aunque el sensor siga detectando inclinación más allá de ±30°.
- Se implementan **filtros digitales** (media móvil o filtro complementario) y **zona muerta** para evitar parpadeos por ruido del sensor.

El sistema se comporta como una **herramienta de medición didáctica**: al inclinar la placa o protoboard, la burbuja se mueve en la misma dirección que la inclinación, de manera suave y coherente, y el LCD o el monitor serial pueden mostrar el ángulo en grados.

Requerimientos

Componente	Descripción	Cantidad
Microcontrolador	ATmega328P (Arduino UNO/Nano)	1
Sensor IMU	MPU6050 (acelerómetro + giroscopio, I2C)	1
Matriz LED	Matriz RGB 8x8 (WS2812B o módulo MAX7219)	1
Driver matriz	MAX7219 / 74HC595 / controlador integrado	1
Botón	Pulsador (hold y calibración)	2 (o 1 con doble función)

Componente	Descripción	Cantidad
Buzzer	Buzzer activo (aviso de límite alcanzado)	1
Pantalla	LCD 16x2 I2C (opcional, para mostrar ángulo)	1
Alimentación	Fuente 5 V regulada	1
Cables	Jumper y protoboard	varios

Funcionalidades principales

1. **Lectura del MPU6050 por I2C:** se obtienen aceleraciones en X, Y, Z y velocidades angulares; se calculan los ángulos de inclinación (pitch y roll) mediante atan2.
2. **Filtrado digital:** se aplica un filtro complementario o media móvil para suavizar la señal y eliminar ruido.
3. **Mapeo ángulo → coordenadas:** los ángulos de ±30° se mapean al rango 0–7 de la matriz, con saturación en los bordes.
4. **Burbuja luminosa:** se enciende el LED correspondiente a la posición calculada, con el color según el eje predominante (rojo X, verde Y, amarillo combinado, azul centrado).
5. **Zona muerta:** si la inclinación es menor a ±3°, la burbuja permanece en el centro y el color es azul, evitando parpadeos.
6. **Límites de la matriz:** la burbuja nunca sale del rango [0..7] × [0..7], aunque la inclinación exceda ±30°.
7. **Modo HOLD:** al presionar el botón HOLD, la burbuja congela su posición y color; se reanuda al volver a presionar.
8. **Calibración:** al presionar el botón CALIBRAR, se toma la posición actual como nuevo "cero" y se guarda el offset en EEPROM para futuras sesiones.
9. **Indicador de límite:** cuando la burbuja toca un borde, el buzzer emite un pitido corto y el LED parpadea para avisar que se alcanzó el límite físico.
10. **Visualización del ángulo (opcional):** el LCD o el monitor serial muestran en tiempo real los ángulos pitch y roll en grados, junto con el estado del modo HOLD.

Informe de Proyecto según Formato IEEE

Componentes del Informe:

1. **Diagrama de Máquinas de Estado:**
    - Representación gráfica detallada de los estados y transiciones del sistema.
    - Descripción de los estados fundamentales.
    - Explicación del flujo de operación y las condiciones de transición entre los estados.
2. **Presentación de la solución programada:**
    - Presentación del código desarrollado en ensamblador para el ATmega328P.
    - Explicación detallada del funcionamiento del código, incluyendo cómo se implementan las transiciones de estado, el manejo de tiempos, y uso de registros.
    - Video y demostración en vivo de la solución.
3. **Implementación del Código en C:**
    - El código será implementado en lenguaje C ajustado a los recursos del ATmega328P.
    - El código será versionado y gestionado utilizando GitHub, proporcionando trazabilidad de al menos tres modificaciones significativas.
4. **Repositorio en GitHub:**
    - El código fuente y sus versiones estarán disponibles en un repositorio público de GitHub.
    - Se documentará el proceso de desarrollo, incluyendo las tres modificaciones principales realizadas durante la implementación.
    - En el informe y la plataforma se incluirá el enlace al repositorio, asegurando la transparencia y trazabilidad del código.

Formato y Presentación del Informe:

- El informe será presentado en el formato IEEE estándar, con secciones bien definidas para la introducción, metodología, resultados, y conclusiones.
- El diagrama de la máquina de estados y el enlace al repositorio GitHub serán incluidos en las secciones correspondientes del informe.
- El informe deberá reflejar un análisis detallado del diseño y desarrollo, así como la justificación técnica de las decisiones tomadas durante el proyecto.

Advertencia: Cualquier indicio de copia del código será calificado automáticamente con 0.